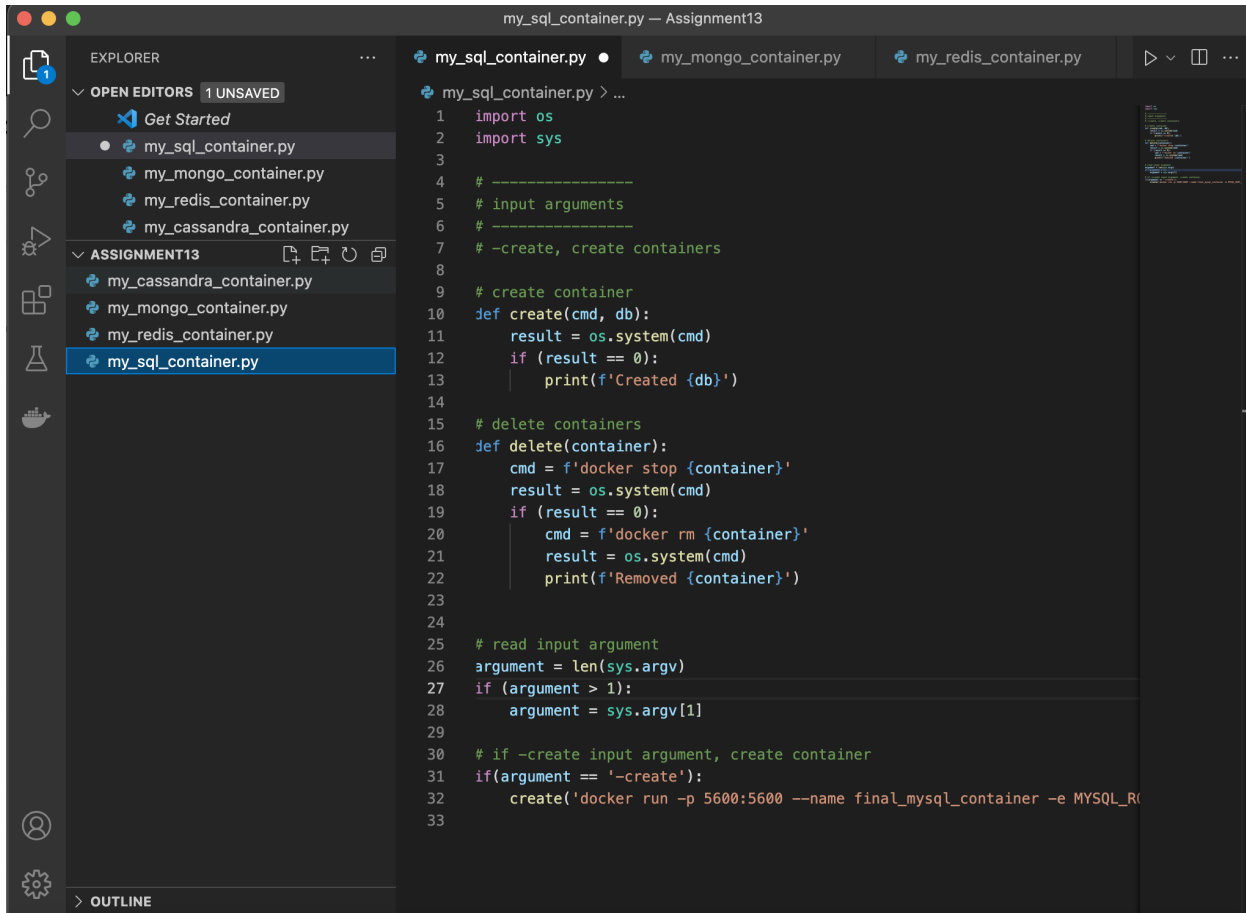


Final Assignment 13.1: Change Data Capture (CDC)

Part 1

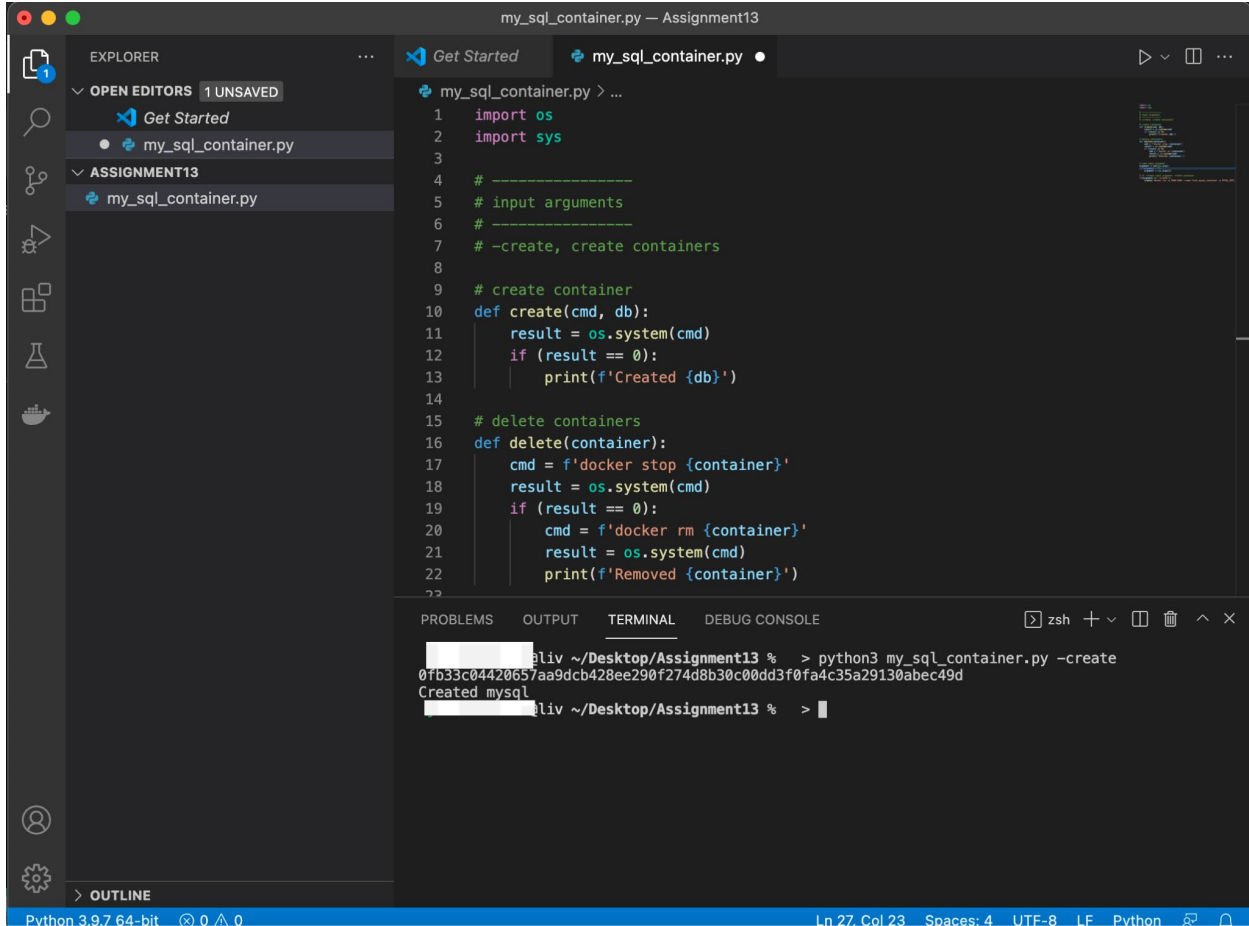
1)



The screenshot shows a Visual Studio Code editor window titled "my_sql_container.py - Assignment13". The Explorer sidebar on the left shows a project named "ASSIGNMENT13" with several Python files. The "my_sql_container.py" file is selected and its code is displayed in the main editor. The code is a Python script that uses the `os` module to interact with the operating system. It defines two functions: `create(cmd, db)` and `delete(container)`. The `create` function runs a system command and prints the result. The `delete` function runs two system commands to stop and remove a container. The script also reads a command-line argument and uses it to call the `create` function.

```
1 import os
2 import sys
3
4 # -----
5 # input arguments
6 # -----
7 # -create, create containers
8
9 # create container
10 def create(cmd, db):
11     result = os.system(cmd)
12     if (result == 0):
13         print(f'Created {db}')
14
15 # delete containers
16 def delete(container):
17     cmd = f'docker stop {container}'
18     result = os.system(cmd)
19     if (result == 0):
20         cmd = f'docker rm {container}'
21         result = os.system(cmd)
22         print(f'Removed {container}')
23
24
25 # read input argument
26 argument = len(sys.argv)
27 if (argument > 1):
28     argument = sys.argv[1]
29
30 # if -create input argument, create container
31 if(argument == '-create'):
32     create('docker run -p 5600:5600 --name final_mysql_container -e MYSQL_R
33
```

2)

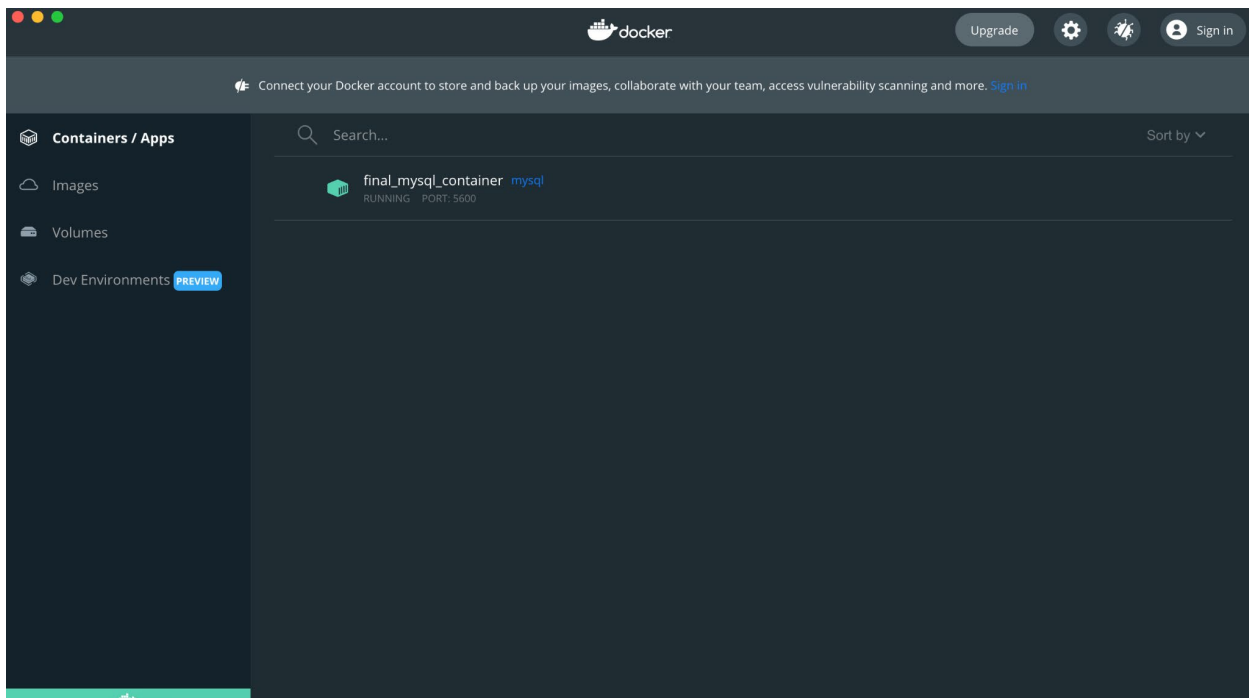


The screenshot shows a VS Code editor window titled "my_sql_container.py — Assignment13". The Explorer sidebar on the left shows the file "my_sql_container.py" under the "ASSIGNMENT13" folder. The main editor area displays the following Python code:

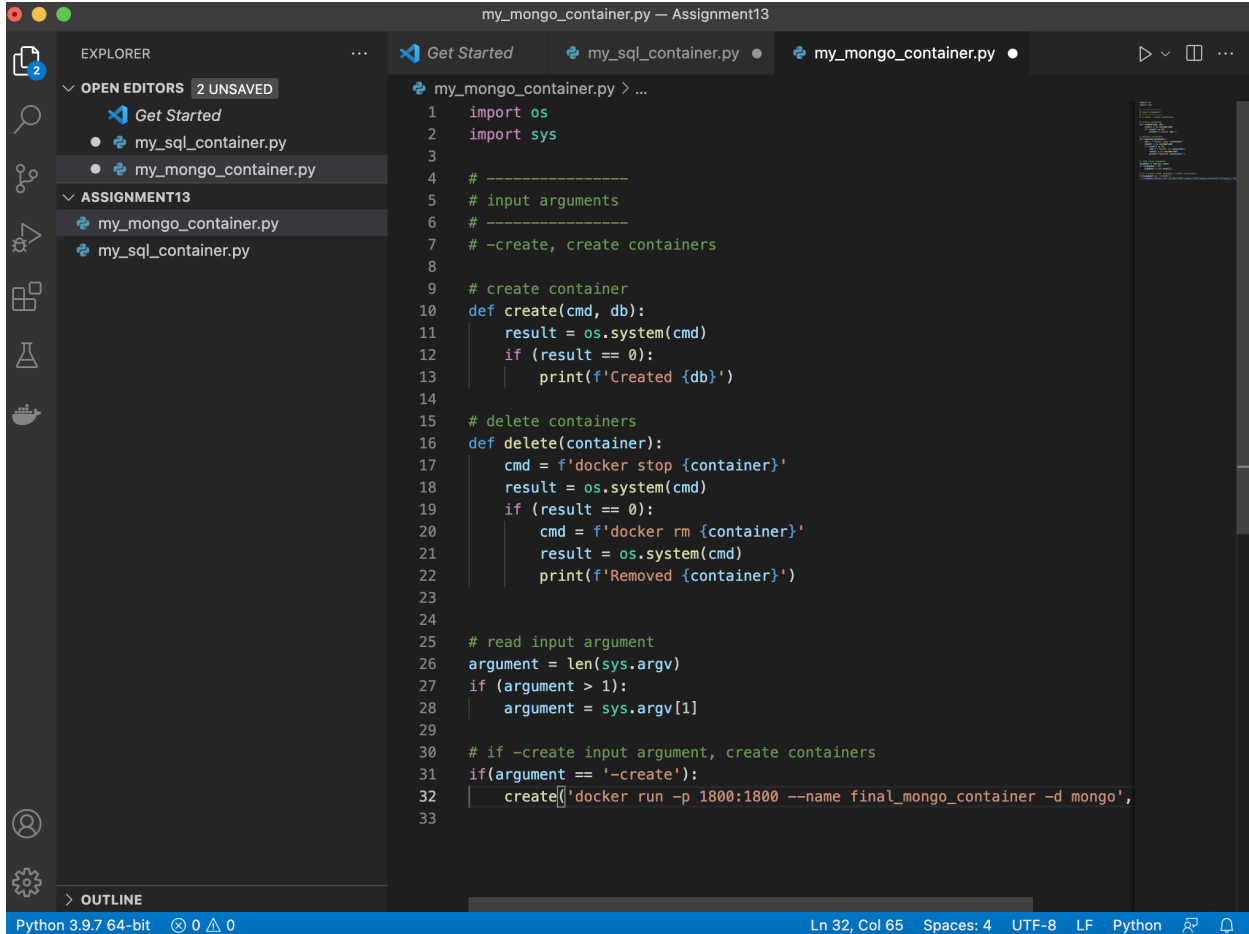
```
1 import os
2 import sys
3
4 # -----
5 # input arguments
6 # -----
7 # -create, create containers
8
9 # create container
10 def create(cmd, db):
11     result = os.system(cmd)
12     if (result == 0):
13         print(f'Created {db}')
14
15 # delete containers
16 def delete(container):
17     cmd = f'docker stop {container}'
18     result = os.system(cmd)
19     if (result == 0):
20         cmd = f'docker rm {container}'
21         result = os.system(cmd)
22         print(f'Removed {container}')
```

The TERMINAL panel at the bottom shows the command prompt output:

```
aliv ~/Desktop/Assignment13 % > python3 my_sql_container.py -create
0fb33c04420657aa9dcb428ee290f274d8b30c00dd3f0fa4c35a29130abec49d
Created mysql
aliv ~/Desktop/Assignment13 % >
```



3)

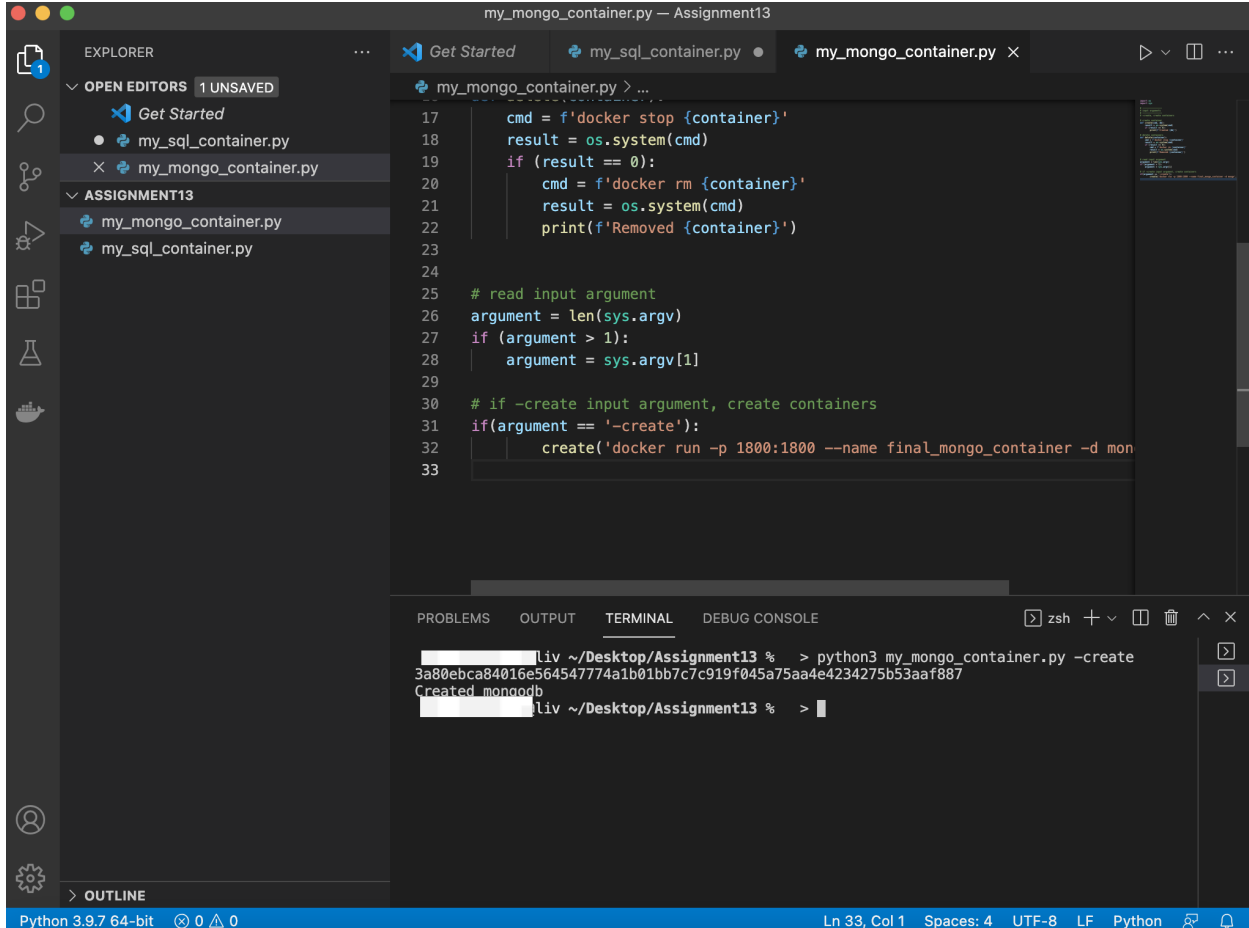


The screenshot shows a Visual Studio Code editor window titled "my_mongo_container.py — Assignment13". The Explorer sidebar on the left shows a project structure with "OPEN EDITORS" (2 UNSAVED) containing "my_sql_container.py" and "my_mongo_container.py", and "ASSIGNMENT13" containing the same two files. The main editor displays the content of "my_mongo_container.py", which is a Python script for managing Docker containers. The script includes imports for 'os' and 'sys', comments for input arguments and container management, and functions for creating and deleting containers. The current line of code is a call to the 'create' function with a Docker run command.

```
1 import os
2 import sys
3
4 # -----
5 # input arguments
6 # -----
7 # -create, create containers
8
9 # create container
10 def create(cmd, db):
11     result = os.system(cmd)
12     if (result == 0):
13         print(f'Created {db}')
14
15 # delete containers
16 def delete(container):
17     cmd = f'docker stop {container}'
18     result = os.system(cmd)
19     if (result == 0):
20         cmd = f'docker rm {container}'
21         result = os.system(cmd)
22         print(f'Removed {container}')
23
24
25 # read input argument
26 argument = len(sys.argv)
27 if (argument > 1):
28     argument = sys.argv[1]
29
30 # if -create input argument, create containers
31 if(argument == '-create'):
32     create(f'docker run -p 1800:1800 --name final_mongo_container -d mongo',
33
```

The status bar at the bottom indicates "Python 3.9.7 64-bit", "Ln 32, Col 65", "Spaces: 4", "UTF-8", "LF", and "Python".

4)



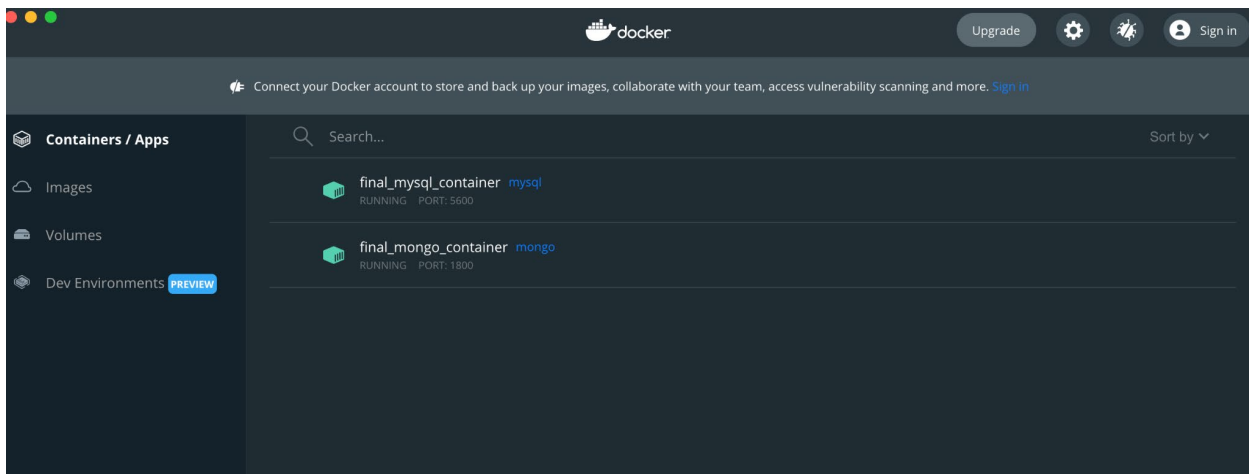
```
my_mongo_container.py — Assignment13
EXPLORER
  OPEN EDITORS 1 UNSAVED
    Get Started
    my_sql_container.py
    my_mongo_container.py
  ASSIGNMENT13
    my_mongo_container.py
    my_sql_container.py

my_mongo_container.py > ...
17 cmd = f'docker stop {container}'
18 result = os.system(cmd)
19 if (result == 0):
20     cmd = f'docker rm {container}'
21     result = os.system(cmd)
22     print(f'Removed {container}')
23
24
25 # read input argument
26 argument = len(sys.argv)
27 if (argument > 1):
28     argument = sys.argv[1]
29
30 # if -create input argument, create containers
31 if(argument == '-create'):
32     create('docker run -p 1800:1800 --name final_mongo_container -d mon
33
```

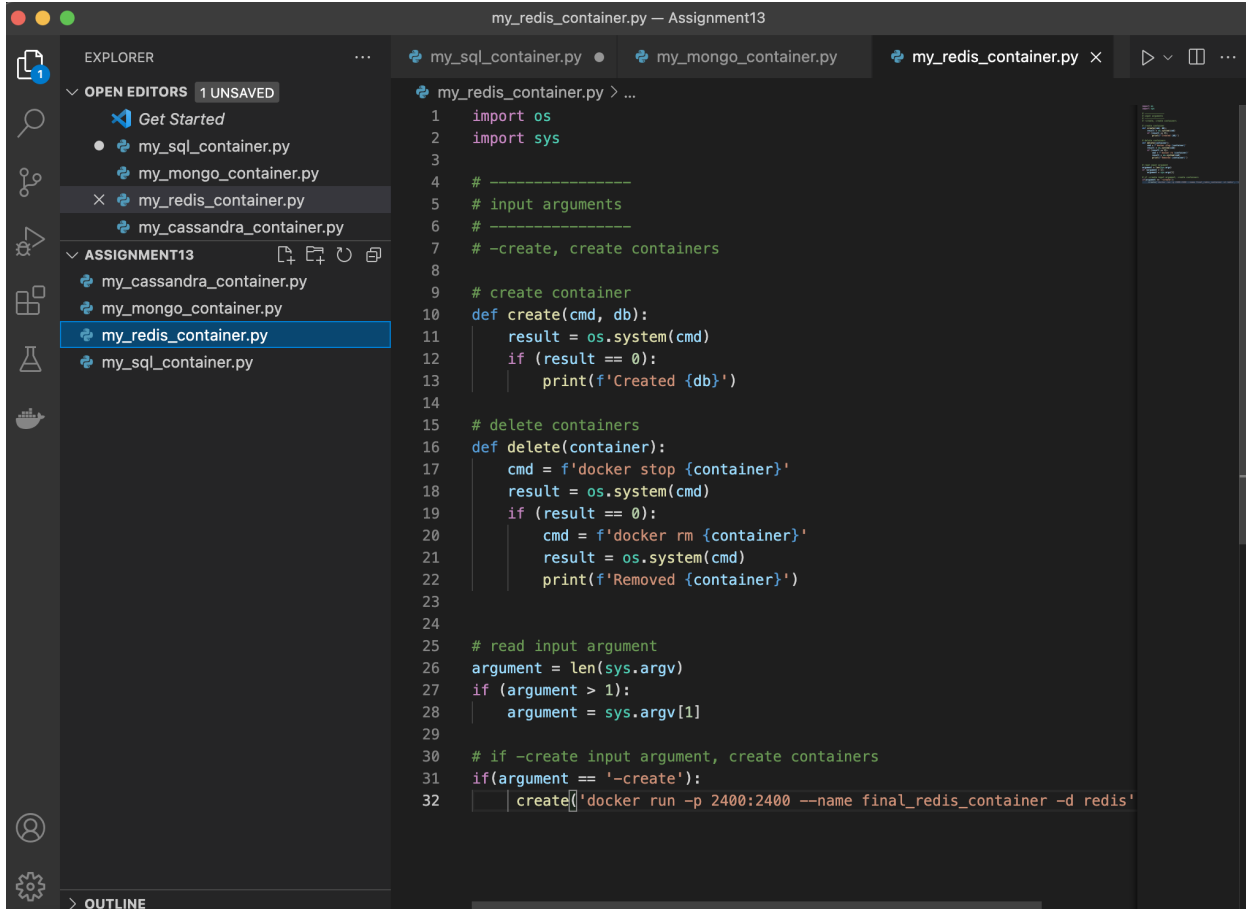
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
zsh
[redacted] liv ~/Desktop/Assignment13 % > python3 my_mongo_container.py -create
3a80ebca84016e564547774a1b01bb7c7c919f045a75aa4e4234275b53aaf887
Created mongod
[redacted] liv ~/Desktop/Assignment13 % >
```

Python 3.9.7 64-bit 0 0 0 Ln 33, Col 1 Spaces: 4 UTF-8 LF Python



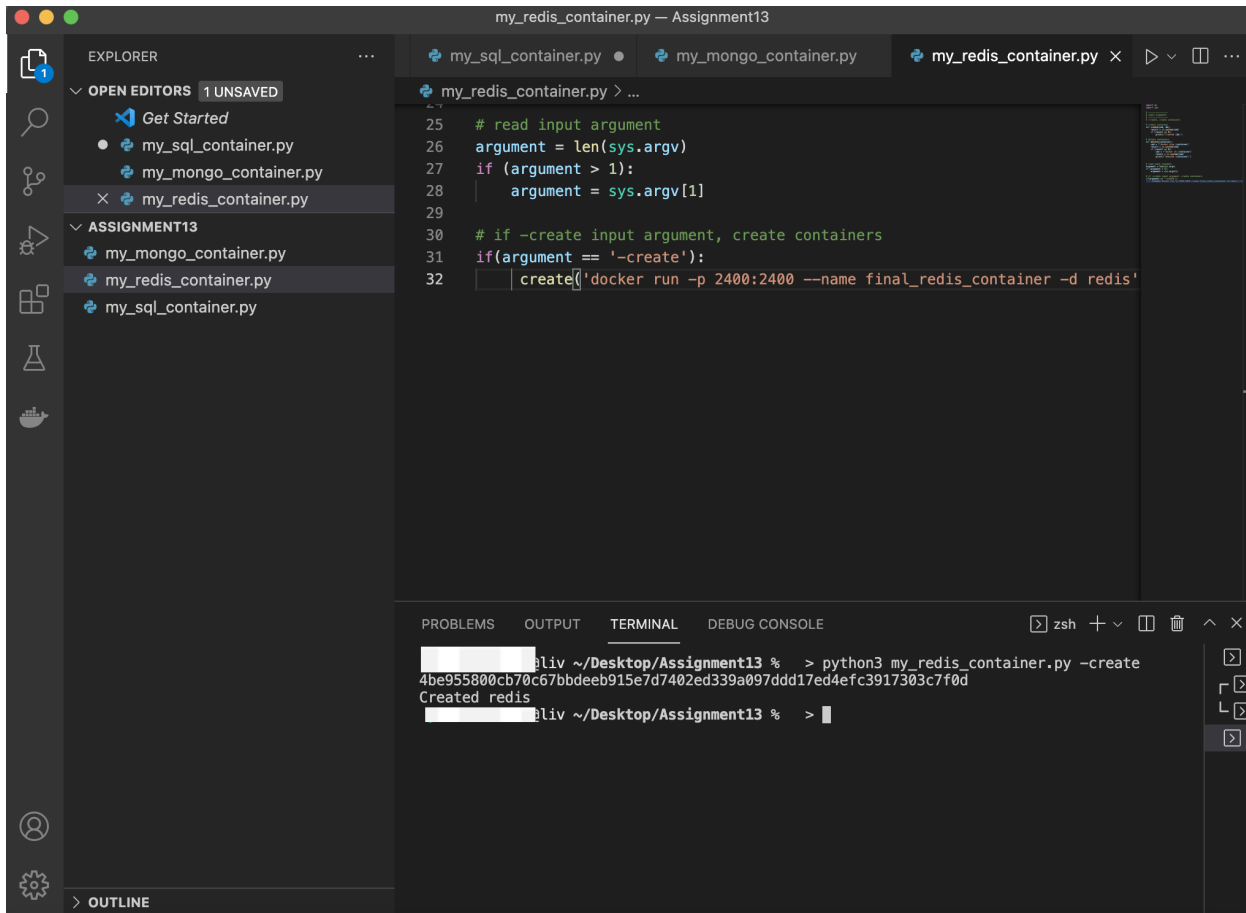
5)



The screenshot shows a Visual Studio Code editor window titled "my_redis_container.py - Assignment13". The Explorer sidebar on the left shows a project named "ASSIGNMENT13" with several files: "my_cassandra_container.py", "my_mongo_container.py", "my_redis_container.py" (selected), and "my_sql_container.py". The main editor displays the code for "my_redis_container.py".

```
1 import os
2 import sys
3
4 # -----
5 # input arguments
6 # -----
7 # -create, create containers
8
9 # create container
10 def create(cmd, db):
11     result = os.system(cmd)
12     if (result == 0):
13         print(f'Created {db}')
14
15 # delete containers
16 def delete(container):
17     cmd = f'docker stop {container}'
18     result = os.system(cmd)
19     if (result == 0):
20         cmd = f'docker rm {container}'
21         result = os.system(cmd)
22         print(f'Removed {container}')
23
24
25 # read input argument
26 argument = len(sys.argv)
27 if (argument > 1):
28     argument = sys.argv[1]
29
30 # if -create input argument, create containers
31 if(argument == '-create'):
32     create('docker run -p 2400:2400 --name final_redis_container -d redis')
```

6)

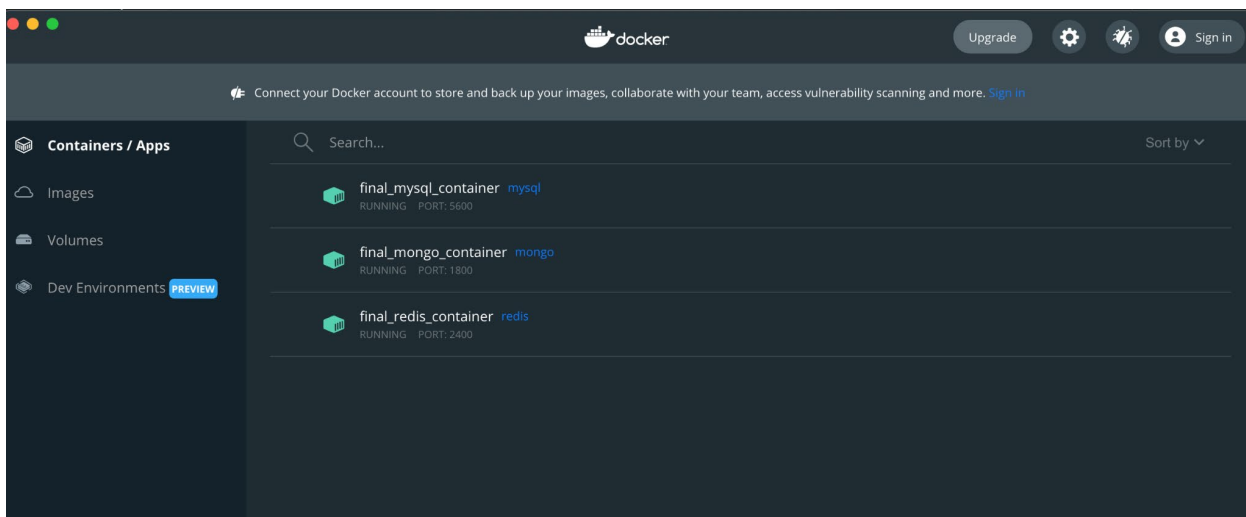


The screenshot shows a VS Code editor window titled "my_redis_container.py — Assignment13". The Explorer sidebar on the left shows the file structure with "my_redis_container.py" selected. The main editor displays the following Python code:

```
25 # read input argument
26 argument = len(sys.argv)
27 if (argument > 1):
28     argument = sys.argv[1]
29
30 # if -create input argument, create containers
31 if(argument == '-create'):
32     create('docker run -p 2400:2400 --name final_redis_container -d redis')
```

The terminal at the bottom shows the command being executed:

```
bliv ~/Desktop/Assignment13 % > python3 my_redis_container.py -create
4be955800cb70c67bbdeeb915e7d7402ed339a097ddd17ed4efc3917303c7f0d
Created redis
bliv ~/Desktop/Assignment13 % >
```



7)

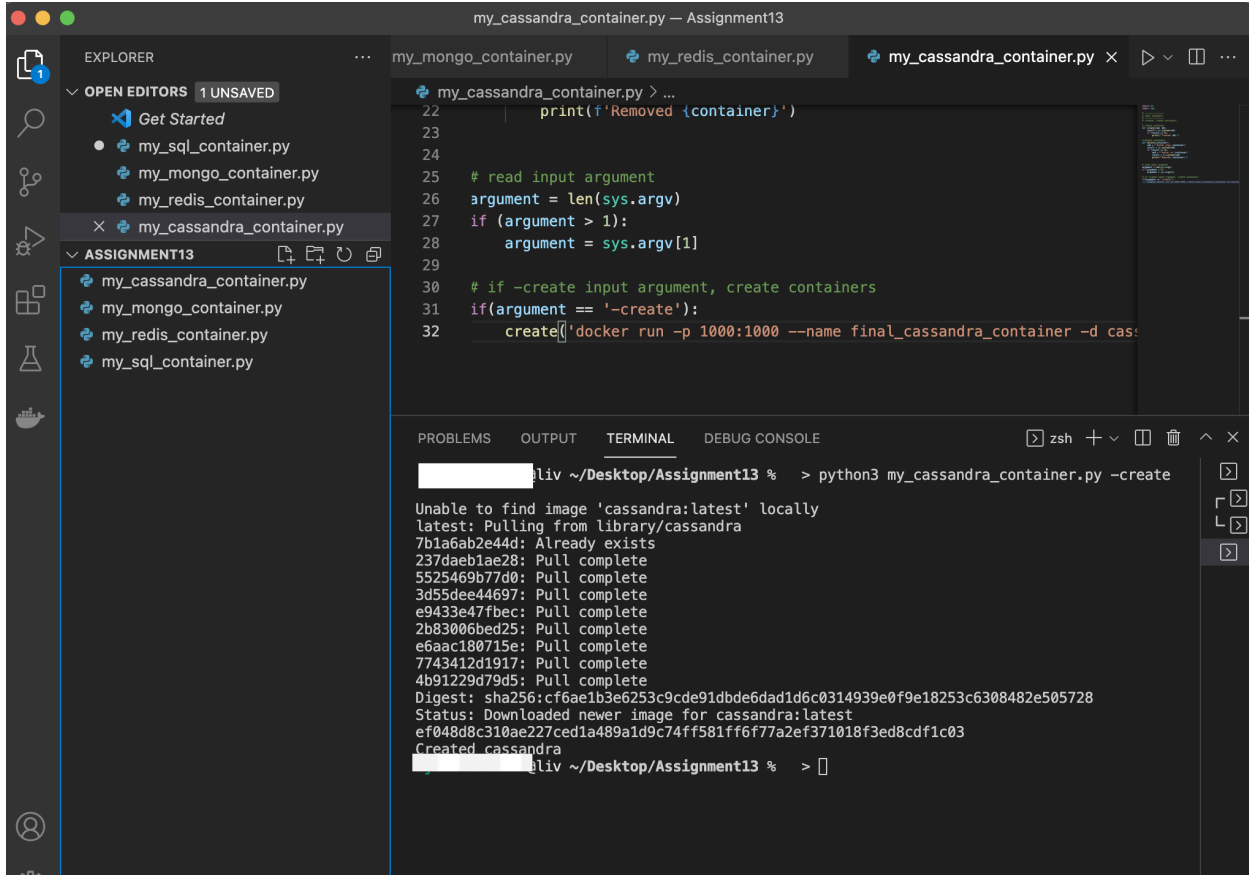
```
my_cassandra_container.py — Assignment13

EXPLORER
  OPEN EDITORS 1 UNSAVED
    Get Started
    my_sql_container.py
    my_mongo_container.py
    my_redis_container.py
    my_cassandra_container.py
  ASSIGNMENT13
    my_cassandra_container.py
    my_mongo_container.py
    my_redis_container.py
    my_sql_container.py

my_cassandra_container.py > ...
1  import os
2  import sys
3
4  # -----
5  # input arguments
6  # -----
7  # -create, create containers
8
9  # create container
10 def create(cmd, db):
11     result = os.system(cmd)
12     if (result == 0):
13         print(f'Created {db}')
14
15 # delete containers
16 def delete(container):
17     cmd = f'docker stop {container}'
18     result = os.system(cmd)
19     if (result == 0):
20         cmd = f'docker rm {container}'
21         result = os.system(cmd)
22         print(f'Removed {container}')
23
24
25 # read input argument
26 argument = len(sys.argv)
27 if (argument > 1):
28     argument = sys.argv[1]
29
30 # if -create input argument, create containers
31 if (argument == '-create'):
32     create(f'docker run -p 1000:1000 --name final_cassandra_container -d cas:')
```

Python 3.9.7 64-bit 0 0 Ln 32, Col 69 Spaces: 4 UTF-8 LF Python

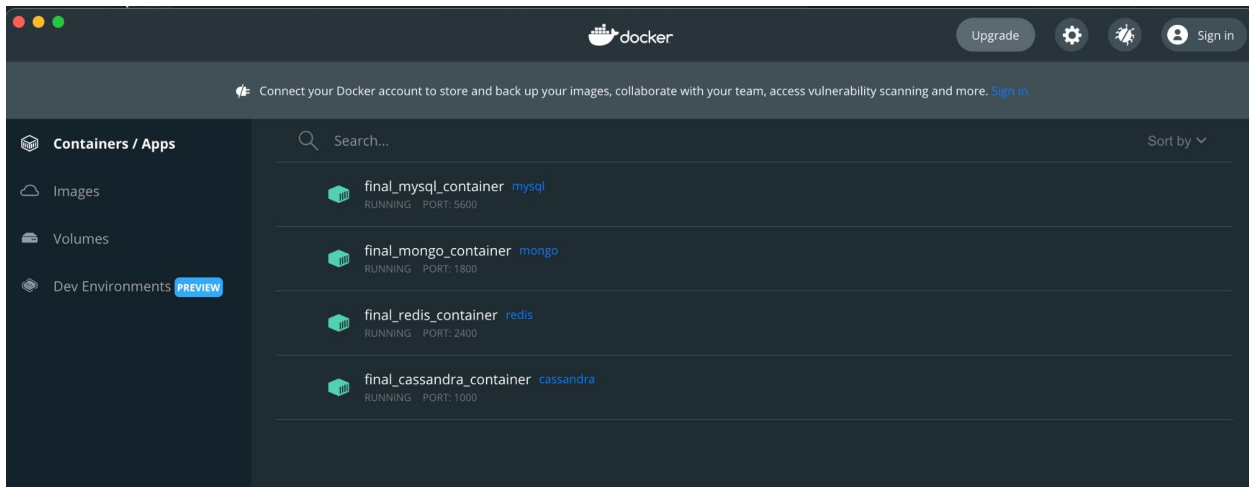
8)



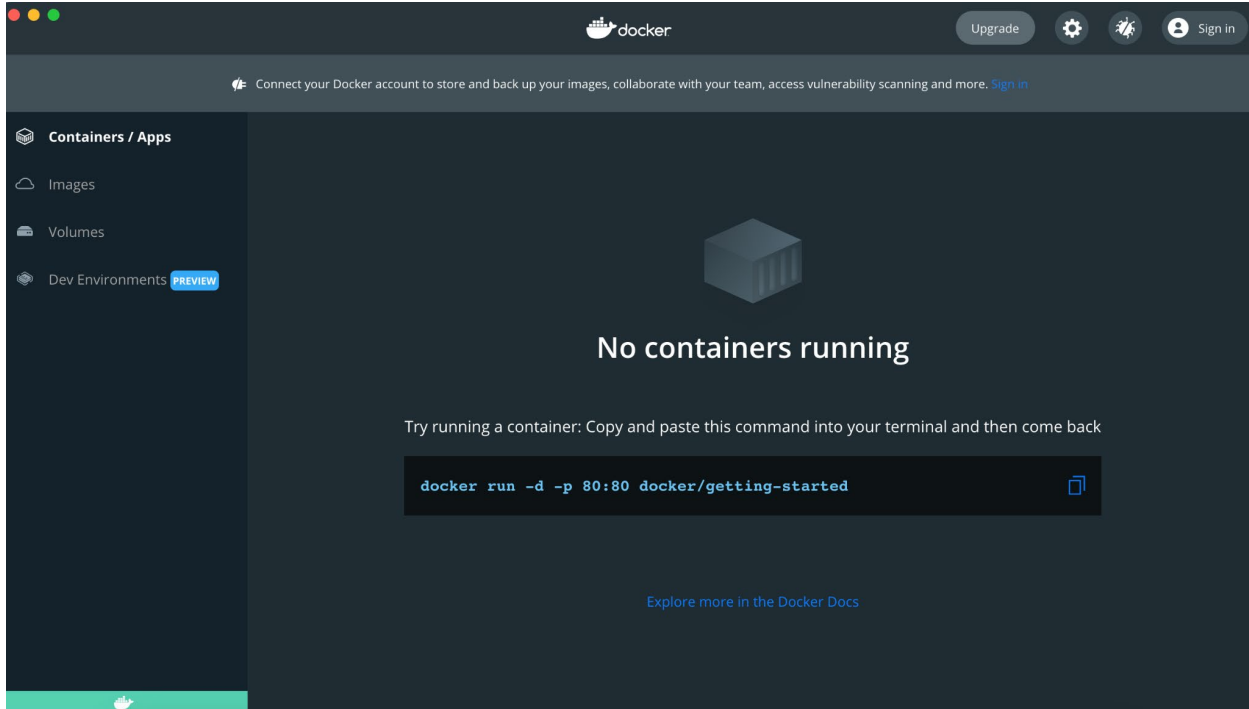
```
my_cassandra_container.py — Assignment13
EXPLORER
  OPEN EDITORS 1 UNSAVED
    Get Started
    my_sql_container.py
    my_mongo_container.py
    my_redis_container.py
    my_cassandra_container.py
  ASSIGNMENT13
    my_cassandra_container.py
    my_mongo_container.py
    my_redis_container.py
    my_sql_container.py

my_cassandra_container.py > ...
22     print(f'Removed {container}')
23
24
25     # read input argument
26     argument = len(sys.argv)
27     if (argument > 1):
28         argument = sys.argv[1]
29
30     # if -create input argument, create containers
31     if(argument == '-create'):
32         create('docker run -p 1000:1000 --name final_cassandra_container -d cas:

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE
@liv ~/Desktop/Assignment13 % > python3 my_cassandra_container.py -create
Unable to find image 'cassandra:latest' locally
latest: Pulling from library/cassandra
7b1a6ab2e44d: Already exists
237daeb1ae28: Pull complete
5525469b77d0: Pull complete
3d55dee44697: Pull complete
e9433e47fbec: Pull complete
2b83006bed25: Pull complete
e6aac180715e: Pull complete
7743412d1917: Pull complete
4b91229d79d5: Pull complete
Digest: sha256:cf6ae1b3e6253c9cde91dbde6dad1d6c0314939e0f9e18253c6308482e505728
Status: Downloaded newer image for cassandra:latest
ef048d8c310ae227ced1a489a1d9c74ff581ff6f77a2ef371018f3ed8cdf1c03
Created cassandra
@liv ~/Desktop/Assignment13 % > 
```

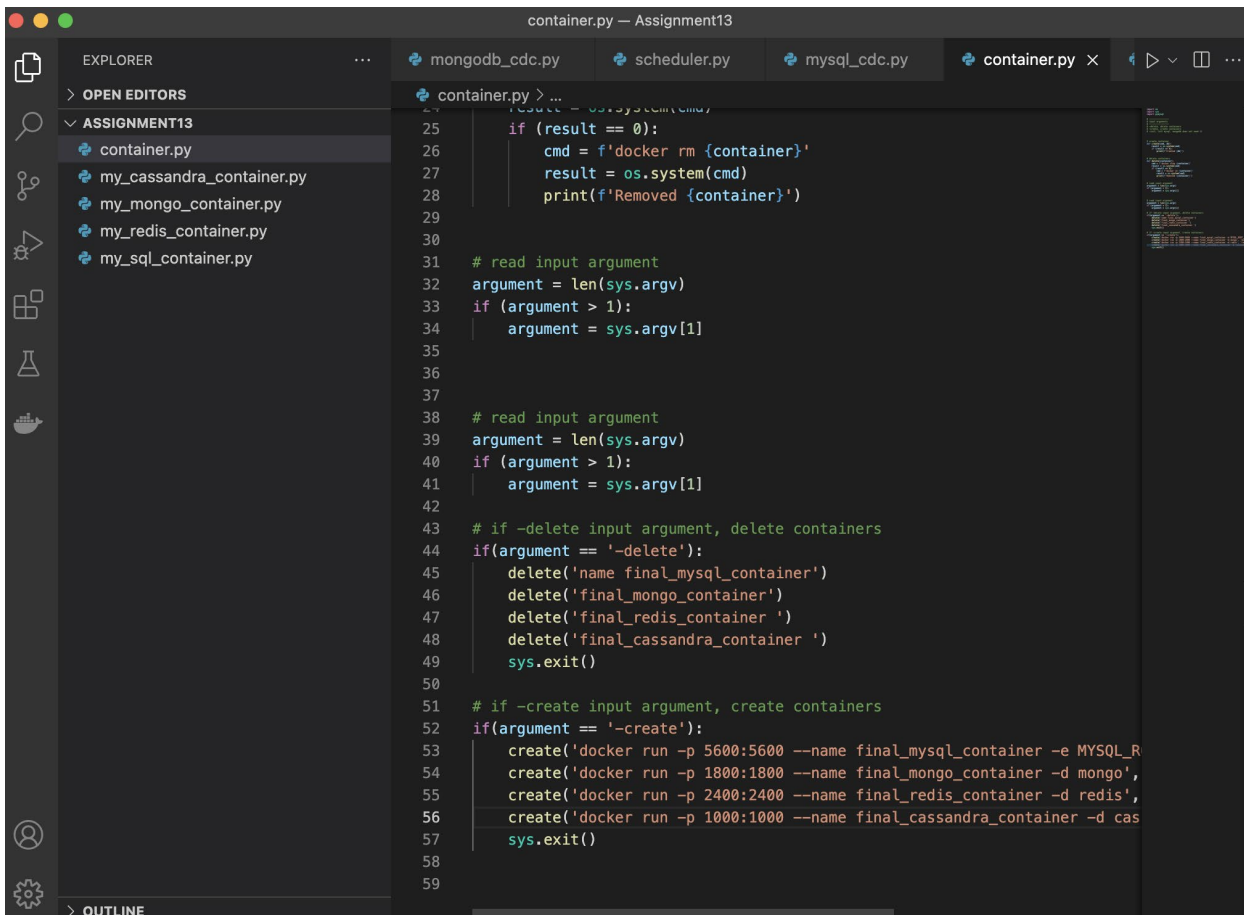


9)



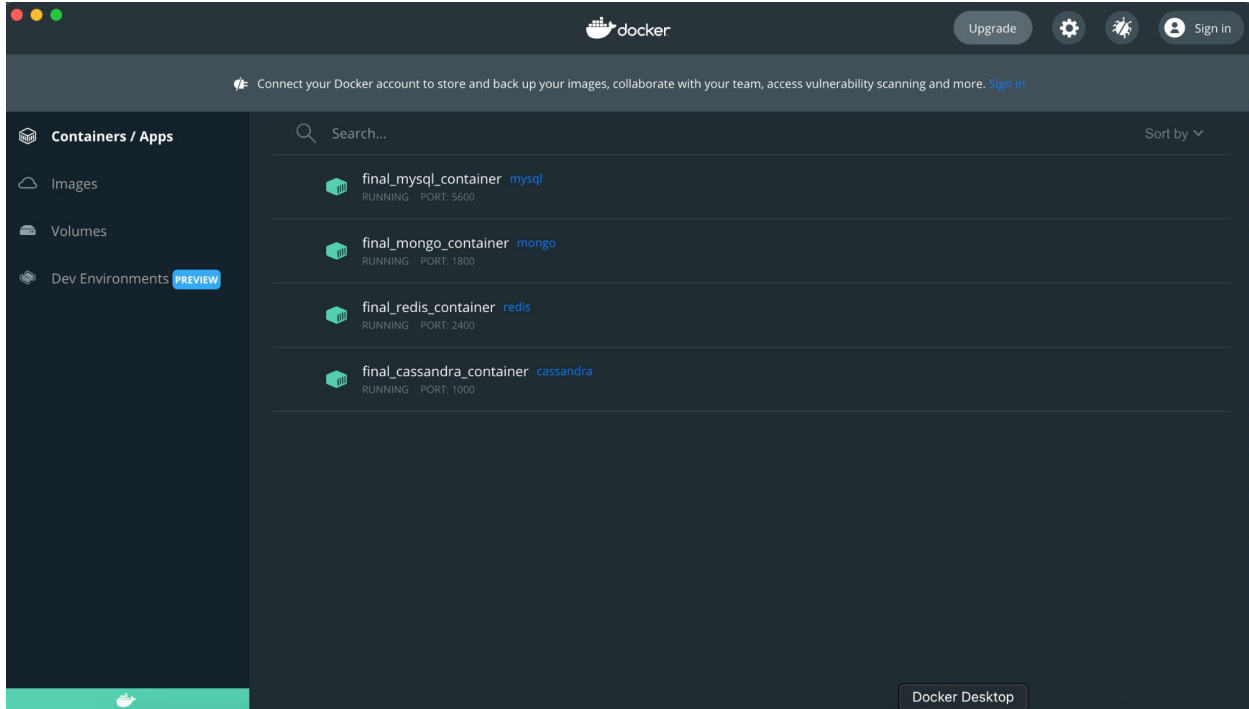
Part 2

10)

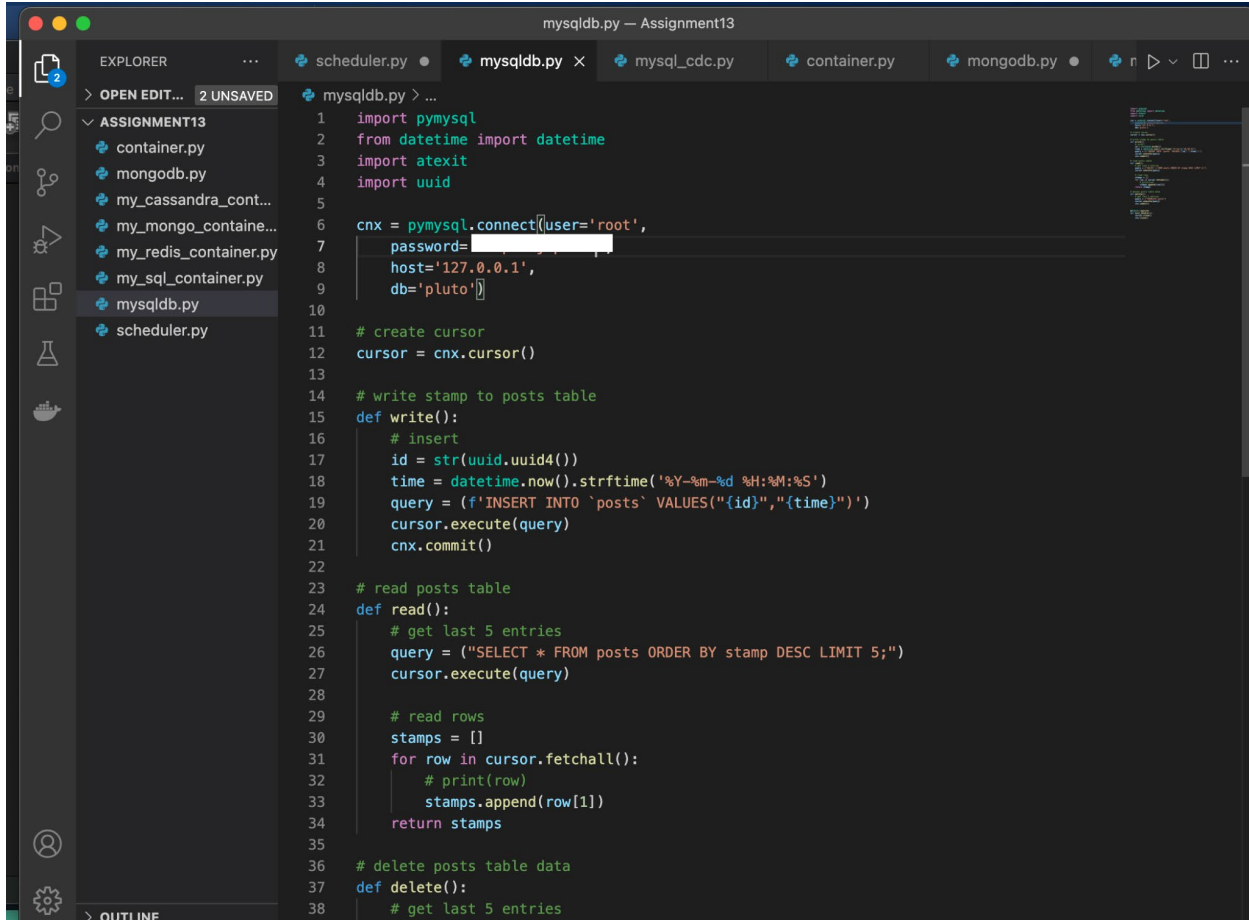


```
container.py — Assignment13
25     if (result == 0):
26         cmd = f'docker rm {container}'
27         result = os.system(cmd)
28         print(f'Removed {container}')
29
30
31 # read input argument
32 argument = len(sys.argv)
33 if (argument > 1):
34     argument = sys.argv[1]
35
36
37
38 # read input argument
39 argument = len(sys.argv)
40 if (argument > 1):
41     argument = sys.argv[1]
42
43 # if -delete input argument, delete containers
44 if(argument == '-delete'):
45     delete('name final_mysql_container')
46     delete('final_mongo_container')
47     delete('final_redis_container ')
48     delete('final_cassandra_container ')
49     sys.exit()
50
51 # if -create input argument, create containers
52 if(argument == '-create'):
53     create('docker run -p 5600:5600 --name final_mysql_container -e MYSQL_R
54     create('docker run -p 1800:1800 --name final_mongo_container -d mongo',
55     create('docker run -p 2400:2400 --name final_redis_container -d redis',
56     create('docker run -p 1000:1000 --name final_cassandra_container -d cas
57     sys.exit()
58
59
```

11)

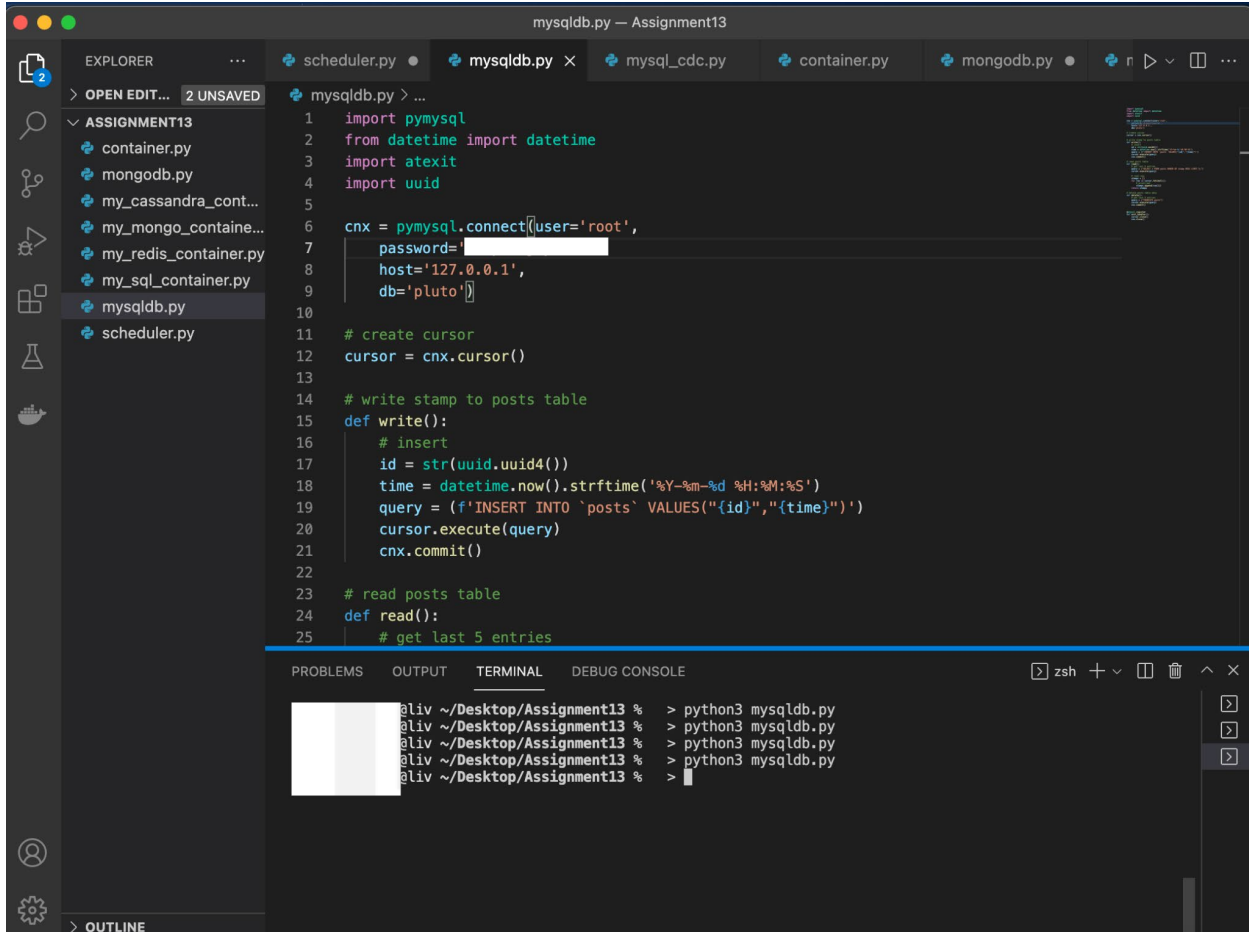


12)



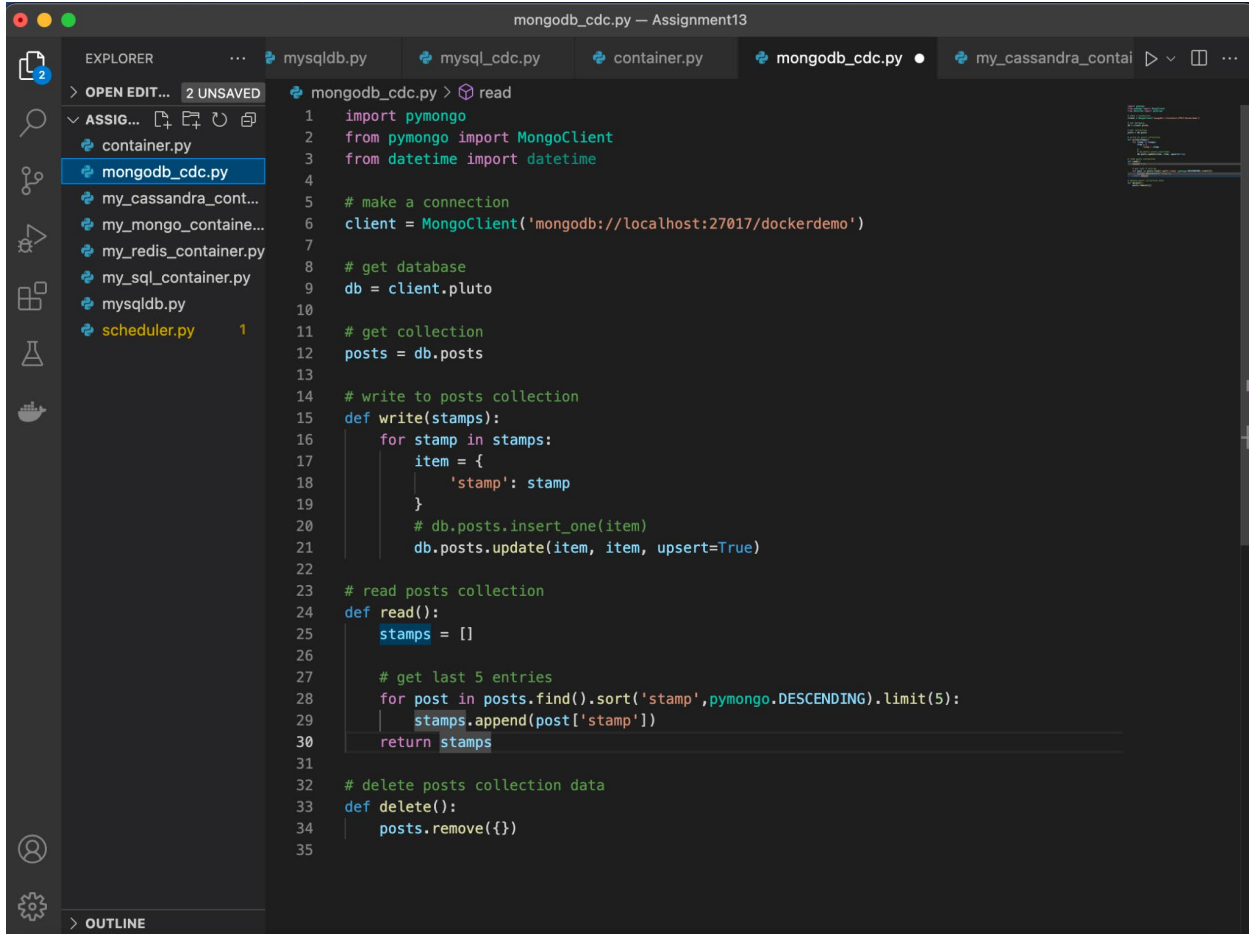
```
mysqlpdb.py — Assignment13
EXPLORER
> OPEN EDIT... 2 UNSAVED
ASSIGNMENT13
  container.py
  mongodb.py
  my_cassandra_cont...
  my_mongo_containe...
  my_redis_container.py
  my_sql_container.py
  mysqlpdb.py
  scheduler.py
mysqlpdb.py > ...
1  import pymysql
2  from datetime import datetime
3  import atexit
4  import uuid
5
6  cnx = pymysql.connect(user='root',
7                        password=
8                        host='127.0.0.1',
9                        db='pluto'])
10
11 # create cursor
12 cursor = cnx.cursor()
13
14 # write stamp to posts table
15 def write():
16     # insert
17     id = str(uuid.uuid4())
18     time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
19     query = (f'INSERT INTO `posts` VALUES("{id}","{time}")')
20     cursor.execute(query)
21     cnx.commit()
22
23 # read posts table
24 def read():
25     # get last 5 entries
26     query = ("SELECT * FROM posts ORDER BY stamp DESC LIMIT 5;")
27     cursor.execute(query)
28
29     # read rows
30     stamps = []
31     for row in cursor.fetchall():
32         # print(row)
33         stamps.append(row[1])
34     return stamps
35
36 # delete posts table data
37 def delete():
38     # get last 5 entries
```

13)



```
mysqladb.py — Assignment13
EXPLORER 2 UNSAVED
  ASSIGNMENT13
    container.py
    mongodb.py
    my_cassandra_cont...
    my_mongo_containe...
    my_redis_container.py
    my_sql_container.py
    mysqladb.py
    scheduler.py
  mysqladb.py > ...
1  import pymysql
2  from datetime import datetime
3  import atexit
4  import uuid
5
6  cnx = pymysql.connect(user='root',
7                        password='pluto',
8                        host='127.0.0.1',
9                        db='pluto')
10
11 # create cursor
12 cursor = cnx.cursor()
13
14 # write stamp to posts table
15 def write():
16     # insert
17     id = str(uuid.uuid4())
18     time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
19     query = (f'INSERT INTO `posts` VALUES("{id}", "{time}")')
20     cursor.execute(query)
21     cnx.commit()
22
23 # read posts table
24 def read():
25     # get last 5 entries
26
27 PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE
28 @liv ~/Desktop/Assignment13 % > python3 mysqladb.py
29 @liv ~/Desktop/Assignment13 % > python3 mysqladb.py
30 @liv ~/Desktop/Assignment13 % > python3 mysqladb.py
31 @liv ~/Desktop/Assignment13 % > python3 mysqladb.py
32 @liv ~/Desktop/Assignment13 % >
```

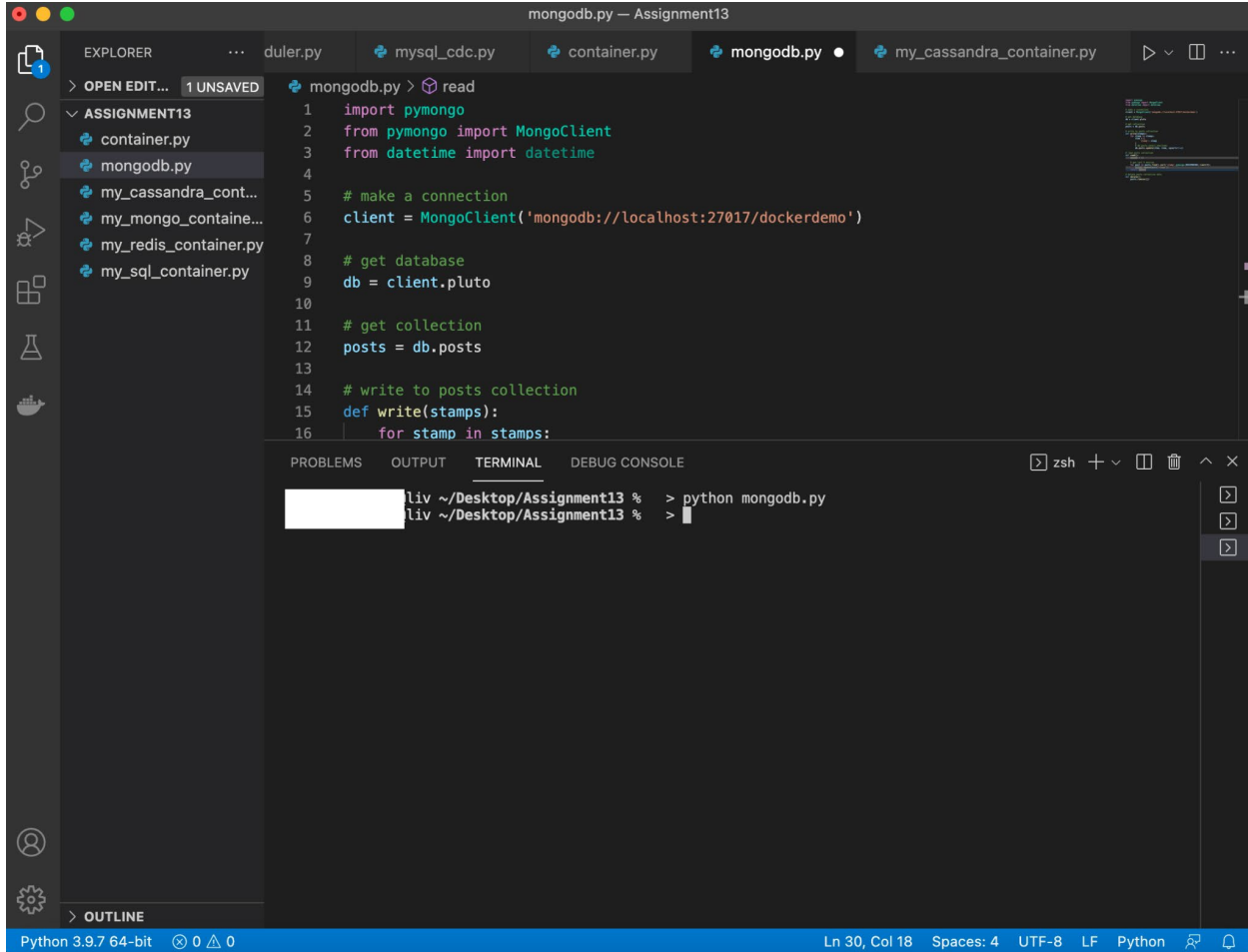
14)



```
mongodb_cdc.py — Assignment13
EXPLORER
  > OPEN EDIT... 2 UNSAVED
  ASSIG...
  container.py
  mongodb_cdc.py
  my_cassandra_cont...
  my_mongo_containe...
  my_redis_container.py
  my_sql_container.py
  mysqlpdb.py
  scheduler.py 1
  > OUTLINE

mongodb_cdc.py > read
1  import pymongo
2  from pymongo import MongoClient
3  from datetime import datetime
4
5  # make a connection
6  client = MongoClient('mongodb://localhost:27017/dockerdemo')
7
8  # get database
9  db = client.pluto
10
11 # get collection
12 posts = db.posts
13
14 # write to posts collection
15 def write(stamps):
16     for stamp in stamps:
17         item = {
18             'stamp': stamp
19         }
20         # db.posts.insert_one(item)
21         db.posts.update(item, item, upsert=True)
22
23 # read posts collection
24 def read():
25     stamps = []
26
27     # get last 5 entries
28     for post in posts.find().sort('stamp', pymongo.DESCENDING).limit(5):
29         stamps.append(post['stamp'])
30     return stamps
31
32 # delete posts collection data
33 def delete():
34     posts.remove({})
35
```

15)



The screenshot shows a Visual Studio Code editor window titled "mongodb.py — Assignment13". The Explorer sidebar on the left shows a project named "ASSIGNMENT13" with several files: "container.py", "mongodb.py", "my_cassandra_cont...", "my_mongo_containe...", "my_redis_container.py", and "my_sql_container.py". The "mongodb.py" file is open in the editor, showing the following Python code:

```
1 import pymongo
2 from pymongo import MongoClient
3 from datetime import datetime
4
5 # make a connection
6 client = MongoClient('mongodb://localhost:27017/dockerdemo')
7
8 # get database
9 db = client.pluto
10
11 # get collection
12 posts = db.posts
13
14 # write to posts collection
15 def write(stamps):
16     for stamp in stamps:
```

Below the editor, the TERMINAL panel is active, showing a shell prompt "zsh" and the command "python mongodb.py" being executed. The terminal output shows the command being run in a shell environment.

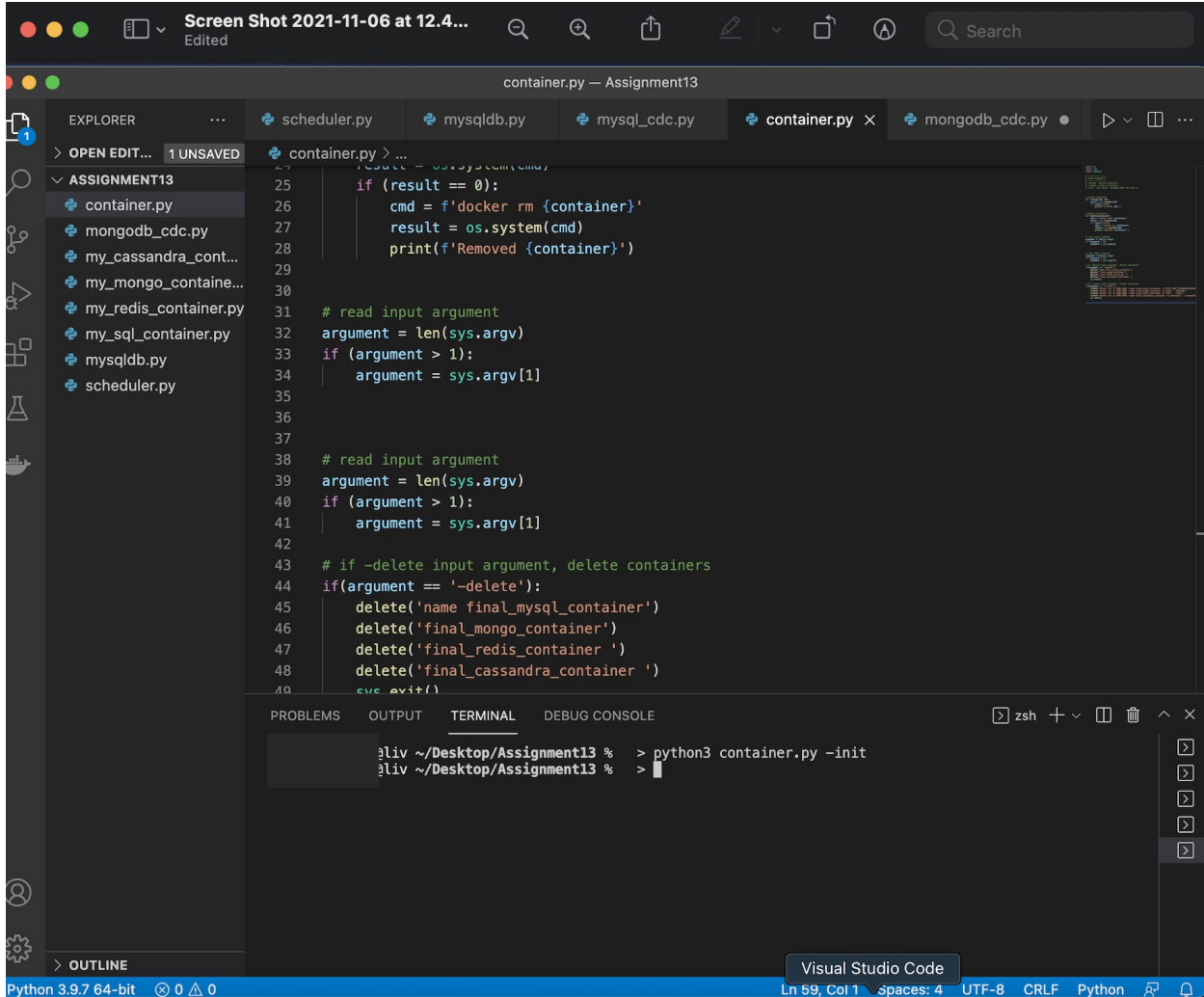
```
lv ~/Desktop/Assignment13 % > python mongodb.py
lv ~/Desktop/Assignment13 % >
```

The status bar at the bottom indicates "Python 3.9.7 64-bit", "Ln 30, Col 18", "Spaces: 4", "UTF-8", "LF", and "Python".

16)

```
1 from threading import Timer
2 import time
3 import mysql
4 import mongodb_cdc
5 import sys
6
7 # delete data in all dbs
8 def clearout():
9     mysql.delete()
10    mongodb_cdc.delete()
11    print('Deleted data in all dbs!')
12
13 # read input argument
14 argument = len(sys.argv)
15 if (argument > 1):
16     argument = sys.argv[1]
17
18 # if -clear input argument, delete data
19 if(argument == '-clear'):
20     clearout()
21     sys.exit()
22
23 # -----
24 # time loop
25 # -----
26
27 def status(stamps,db):
28     print(f'Data in {db}:')
29     for stamp in stamps:
30         print(stamp)
31     time.sleep(2)
32
33 def mysql():
34     mysql.write()
35
36 def mongo():
37     stamps = mysql.read()
38     status(stamps,'mysql')
39     mongodb_cdc.status(stamps)
```


17)



Screen Shot 2021-11-06 at 12.4... Edited

container.py — Assignment13

EXPLORER

- OPEN EDIT... 1 UNSAVED
- ASSIGNMENT13
 - container.py
 - mongodb_cdc.py
 - my_cassandra_cont...
 - my_mongo_containe...
 - my_redis_container.py
 - my_sql_container.py
 - mysqldb.py
 - scheduler.py

container.py

```
24 result = os.system(cmd)
25 if (result == 0):
26     cmd = f'docker rm {container}'
27     result = os.system(cmd)
28     print(f'Removed {container}')
29
30
31 # read input argument
32 argument = len(sys.argv)
33 if (argument > 1):
34     argument = sys.argv[1]
35
36
37
38 # read input argument
39 argument = len(sys.argv)
40 if (argument > 1):
41     argument = sys.argv[1]
42
43 # if -delete input argument, delete containers
44 if(argument == '-delete'):
45     delete('name final_mysql_container')
46     delete('final_mongo_container')
47     delete('final_redis_container ')
48     delete('final_cassandra_container ')
49 sys.exit()
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

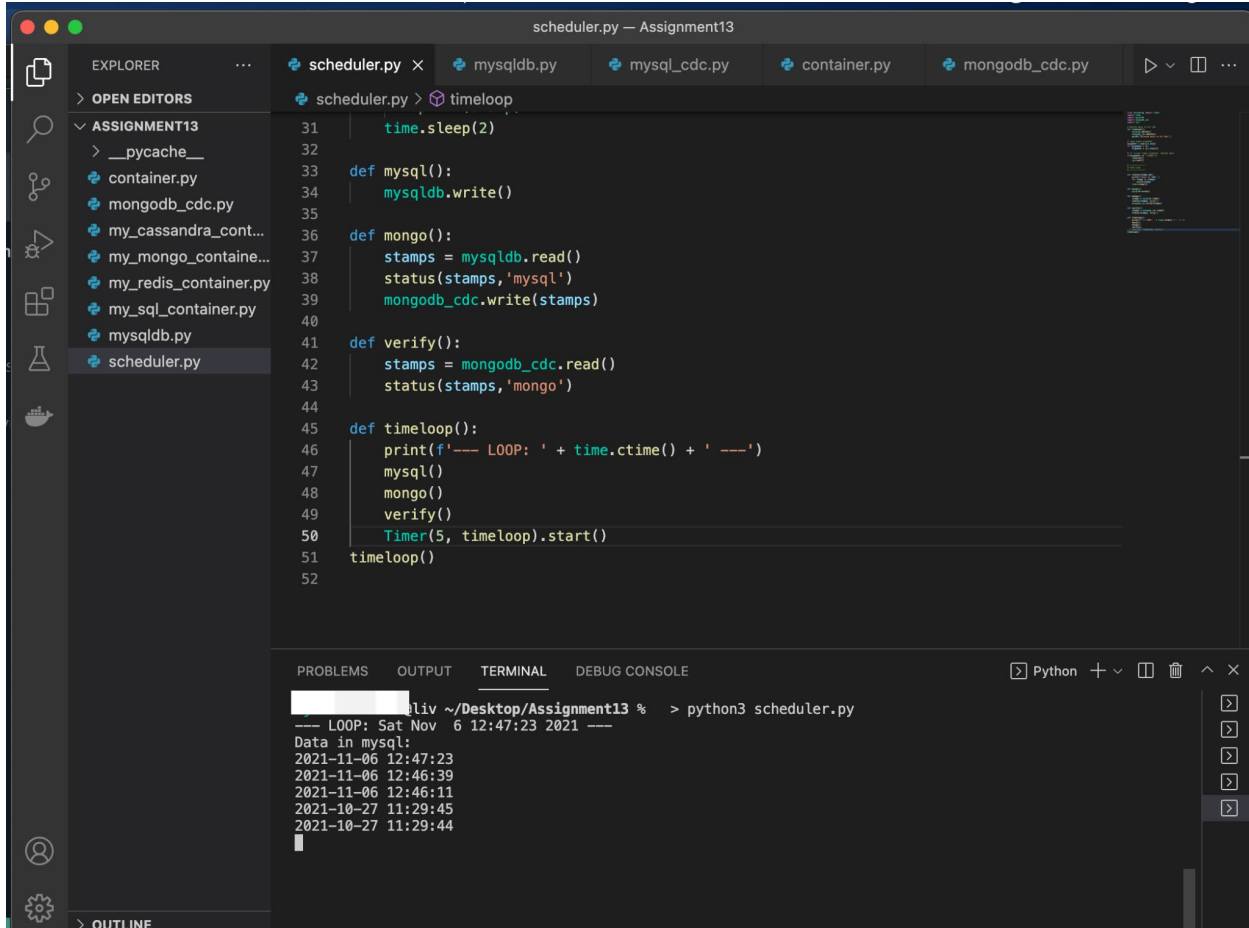
zsh + v

```
~liv ~/Desktop/Assignment13 % > python3 container.py -init
~liv ~/Desktop/Assignment13 % >
```

Visual Studio Code

Python 3.9.7 64-bit 0 0 0 Ln 59, Col 1 \spaces: 4 UTF-8 CRLF Python

18)



```
scheduler.py -- Assignment13
EXPLORER
> OPEN EDITORS
  scheduler.py x mysqlldb.py mysql_cdc.py container.py mongodb_cdc.py
  scheduler.py > timeloop
  31 time.sleep(2)
  32
  33 def mysql():
  34     mysqlldb.write()
  35
  36 def mongo():
  37     stamps = mysqlldb.read()
  38     status(stamps,'mysql')
  39     mongodb_cdc.write(stamps)
  40
  41 def verify():
  42     stamps = mongodb_cdc.read()
  43     status(stamps,'mongo')
  44
  45 def timeloop():
  46     print(f'--- LOOP: ' + time.ctime() + ' ---')
  47     mysql()
  48     mongo()
  49     verify()
  50     Timer(5, timeloop).start()
  51 timeloop()
  52

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE
Python + - [ ] [ ] [ ] [ ]
~ liv ~/Desktop/Assignment13 % > python3 scheduler.py
--- LOOP: Sat Nov  6 12:47:23 2021 ---
Data in mysql:
2021-11-06 12:47:23
2021-11-06 12:46:39
2021-11-06 12:46:11
2021-10-27 11:29:45
2021-10-27 11:29:44
```